

Bug & Usability Findings Log: Scenario D (Lê Phạm Kiều Duyên, 23127184)

§7 mandatory aggregated file. §7 requires every finding recorded here to be submitted to the Google Form (<https://forms.gle/CJQFQCAXcsDbXDMM9>) from the student-ID email, and requires this file and the form submissions to remain consistent, since the TA may cross-check the counts.

Status: 24 findings logged, all 24 submitted to the Google Form. 19 went in on **2026-08-02** — 16 from Task 1B (found 2026-07-31) and 3 from Task 3. The remaining five went in on **2026-08-03**: D-023, raised while clearing the last unexecuted Task 1B cells, and D-024...D-027 from the five Task 2 usability sessions. **The log and the form now agree at 24 each**, which is what §7 asks for and what the TA may cross-check. The Task 1B pass covered all 60 checklist items × all 6 screens: D1 to D4, plus D5 (Notifications) and D6 (the attachment lightbox), the latter two added once the group agreed the scenario's screen set was not fixed at four. Task 3 added D-020 (admin area has no phone-width breakpoint), D-021 (Safari 15 drops the whole stylesheet) and D-022 (Safari 16 never renders the sign-in inputs). **Three findings were retracted before submission: D-013, D-014 and D-018.** D-014 fell on a self-review of the log; D-013 and D-018 fell on a live re-verification against EMS on 2026-07-31, which also narrowed D-016 to a single component. The three retraction notes sit below the findings table, and the reasoning is set out under "Live re-verification" in

`docs/02_Task1B_Execution_Report_ScenarioD.md`. **Task 2 ran on 2026-08-03** with five real participants and contributed D-024...D-027, plus independent user-side confirmation of D-015.

Totals

Source	Bug: Blocker	Bug: Critical	Bug: Major	Bug: Minor	Bug: Trivial	Usability 4	Usability 3	Usability 2	Usability 1	Usability 0	Total
Task 1B (checklist execution)	0	0	6	6	1	0	2	2	0	0	17
Task 2 (usability sessions)	0	0	0	0	0	0	2	2	0	0	4
Task 3 (cross-platform)	0	1	2	0	0	0	0	0	0	0	3
Grand total	0	1	8	6	1	0	4	4	0	0	24

Sixteen bugs (1 Critical, 8 Major, 6 Minor, 1 Trivial) and eight usability findings (four at severity 3, four at severity 2). The usability half doubled when the five Task 2 sessions ran on 2026-08-03: **D-024...D-027** all come from what real participants did, and none of them was visible to checklist execution, because each is about a user failing to *find* something that works correctly once found.

Two findings carry no image file, stated here rather than left to be noticed. D-011 (clicking "Export Excel" produces no visible feedback) and D-012 (Enter does nothing in either field of the create-request form) are both defects whose symptom is the *absence* of an on-screen change, so a still capture taken before and after the action would be identical and would evidence nothing. Each cell says so and cites what was actually observed instead: a Network-tab request succeeding with HTTP 200 while the page stays inert, for D-011, and a keystroke-level observation for D-012. A screen recording is the capture that would close both, and none was taken this session. Every other finding names a file that exists under `reports/evidence_task1b/`.

Findings

ID	Scenario/Screen	Type	Description	Expected vs actual	Steps/Heuristic	Severity	Suggested fix	Evidence (screenshot ref)	Form-submission timestamp
D-001	D1, <code>/complaints/new</code>	Bug	Selecting a value in the "Request type" dropdown and then clicking elsewhere within roughly the next second intermittently discards the selection (reverting to the placeholder and showing "Vui lòng chọn một mục trong danh sách") or silently swaps it for a different option the user never clicked, without the user reopening the dropdown or taking any dropdown action.	Expected: a value chosen in "Request type" stays chosen when the user clicks the next field. Actual: the value reverts to the unselected placeholder, or silently changes to an option the user never clicked.	1. Log in as a student, go to <code>/complaints/new</code> . 2. Click the "Request type" combobox, click "Support". 3. Immediately (within ~0.5 s) click into the "Issue requiring support" field below. 4. Observe: Request type reverts to the unselected placeholder and the required-field error appears (or the value silently changes to a different option), even though no dropdown interaction occurred in step 3. Reproduced twice this way; did not reproduce when a ~2 s pause was inserted between steps 2 and 3, consistent with the dropdown's closing/blur transition still intercepting the next click for a short window. Related checklist items IA02-01 (required-field marking) and IA02-09 (submit blocked with a clear summary), though neither is a clean match, see <code>docs/02_Task1B_Execution_Report_ScenarioD.md</code> §"Findings raised" for why this is logged standalone.	Major	Ensure the dropdown's option list is not pointer-interactive (or has released focus/blur handling) during its close transition, so a click immediately after selection lands on the intended next element rather than being absorbed by the closing listbox; and make sure the selected value is committed to form state synchronously on click, not after an animation-dependent timeout.	D1_IA02-01_requesttype_selected_before_blur.jpg, D1_IA02-01_requesttype_reset_after_blur.jpg	2026-08-02
D-002	D1, <code>/complaints/new</code>	Bug	Attachment-rejection messages always name the rule broken but never the offending filename, across all three violation types the item requires testing: "Chỉ chấp nhận ảnh JPG, PNG, GIF và WEBP." (wrong type), "Mỗi ảnh phải có dung lượng không quá 5 MB." (oversize), "Bạn chỉ có thể tải lên tối đa 5 ảnh." (over count). One root cause (the rejection-toast component is never passed the filename), three instances, logged once per the merge rule.	Expected: the rejection message names the rule broken and the offending filename. Actual: it names only the rule, for all three violation types.	1. Log in as a student, go to <code>/complaints/new</code> , scroll to Attachments. 2. Upload a <code>.pdf</code> → message names the rule, not the file. 3. Reload, upload a 6 MB <code>.jpg</code> → same pattern. 4. Reload, upload 6 valid <code>.png</code> files at once → same pattern. Checklist item IA02-01/IA02-04 expects "a message naming the rule broken and the offending filename ".	Major	Pass the rejected <code>File</code> object's <code>.name</code> into the toast/banner component and interpolate it into all three existing message strings (e.g. "big.jpg, mỗi ảnh phải có dung lượng không quá 5 MB.").	D1_IA02-04_reject_messages_no_filename.jpg	2026-08-02

ID	Scenario/Screen	Type	Description	Expected vs actual	Steps/Heuristic	Severity	Suggested fix	Evidence (screenshot ref)	Form-submission timestamp
D-003	D1 /complaints/new, D4 /dashboard/admin/complaints/{id}	Usability	Submitting the create-request form completely empty shows exactly one generic red banner ("Vui lòng nhập loại yêu cầu, vấn đề cần hỗ trợ và mô tả chi tiết.") positioned just above the Submit button, listing all three missing fields together, never an inline message beside each individual offending field. Second instance found on D4 (2026-08-02), and it is worse there. Clicking "Send response" with the response empty does produce a <code>role="alert"</code> message naming the missing field ("Enter the response content before sending.", 93 ms after the click), but it renders as a full-width banner at the top of the page : measured in the DOM, its bounding box sits at <code>y = -234</code> , i.e. above the viewport, while the Response textarea is at <code>y = 235</code> and the Send button at <code>y = 268</code> . From the scroll position the action is necessarily taken at, the feedback is off-screen, so the click reads as a no-op. One root cause (page-level validation banner instead of per-field inline errors), two screen instances, logged once per the merge rule.	Expected: an inline message beside each offending field, in plain language. Actual: one generic banner above the Submit button listing all three fields together, and nothing beside any field.	IA02-08 (Nielsen H9 / error-message placement): "The error appears inline beside the offending field in plain language, not only as one generic banner at the top."	3	Move the per-field validity state (already computed, since native <code>:invalid</code> is set on all three fields) into a small inline <code><p></code> under each offending field, keeping the summary banner as a secondary aid, not the only signal.	D1_IA02-08_generic_banner_not_inline.jpg	2026-08-02
D-004	D1 /complaints/new, D4 /dashboard/admin/complaints/{id}	Bug	Typing text into "Detailed description" and then clicking the "← Quay lại" (Back) link navigates straight to <code>/complaints</code> with no unsaved-changes warning of any kind; the typed text is silently discarded. Second instance found on D4 (2026-08-02) : with unsaved text in the admin "Response content" textarea, clicking "← Back" navigates straight to the request list. A <code>window.confirm</code> interceptor installed before the click recorded nothing, no warning node was added to the DOM, and the draft is gone. One root cause (no unsaved-changes guard on the router transition), two screen instances, logged once per the merge rule.	Expected: an unsaved-changes confirmation before navigating away. Actual: immediate navigation to <code>/complaints</code> , typed text discarded with no warning.	1. Log in as a student, go to <code>/complaints/new</code> . 2. Type any text into "Detailed description". 3. Click "← Quay lại" without saving. 4. Observe: immediate navigation to <code>/complaints</code> , no confirm dialog, text is gone.	Minor	Add a <code>beforeunload /router-guard</code> check: if any of the three text fields is non-empty and the form has not been submitted, show a "Discard changes?" confirm dialog before navigating away via any of the exit paths (Back link, sidebar, browser Back).	D1_IA02-13_back_no_warning_data_loss.jpg	2026-08-02

ID	Scenario/Screen	Type	Description	Expected vs actual	Steps/Heuristic	Severity	Suggested fix	Evidence (screenshot ref)	Form-submission timestamp
D-005	D2 /complaints/ D3 /dashboard/admin/complaints	Bug	The free-text search box ("Tìm theo tiêu đề hoặc mô tả" on D2, "Search name, email or title" on D3, the same shared component) drops keystrokes under a normal fast typing sequence. On D2, typing "Technology" left <code>input.value</code> (confirmed in DevTools) holding only the last character, "y". On D3, typing "Technology" left the field completely empty (a stronger manifestation of the same defect: 0 characters survived, not 1). Both screens: typing the same characters with an explicit pause after each key accumulates correctly, and a second fast attempt on D3 (typing "Technology Day") succeeded completely, confirming the bug is genuinely intermittent/timing-dependent, not a hard 100%-reproducible break. One shared-component root cause, two screen instances, logged once per the merge rule, same family as D-001.	Expected: every character typed appears in the search box. Actual: characters are dropped under fast typing (only the last one survived on D2, none at all on D3).	1. Log in as a student (D2) or admin (D3). 2. Click the search box, type a multi-character term at normal/fast typing speed. 3. Inspect <code>input.value</code> in DevTools: it holds fewer characters than typed (sometimes zero, sometimes only the last one). 4. Repeat with an explicit pause after each keystroke: value accumulates correctly every time observed. 5. Repeat the fast attempt again: sometimes succeeds fully, sometimes fails, confirming intermittency.	Major	Audit the shared search-box component's <code>onChange</code> /debounce handler for a stale-closure bug, a handler reading a previous render's state instead of the latest keystroke produces exactly this "some/all early keys lost" pattern under fast input; needs <code>useState</code> functional updates or a ref-backed debounce instead of a naive <code>setTimeout</code> capturing a stale closure.	D2_search_box_loses_keystrokes.jpg, D3_search_box_lost_all_chars.jpg	2026-08-02
D-006	D2, /complaints/{id}	Bug	Pasting a non-existent support-request id directly (<code>/complaints/999999</code>) renders a correctly-structured "not found" state (icon + message, no blank screen/crash) but with the wrong copy : "Event review not found.", text that belongs to an unrelated feature (event reviews), evidently reused without updating for the complaints route.	Expected: a not-found message naming this route, e.g. "Support request not found." Actual: "Event review not found.", copy belonging to an unrelated feature.	1. While logged in (student or admin), navigate directly to <code>/complaints/999999</code> . 2. Observe the red "!" icon and the message "Event review not found.", contrast with the correct-context messages used elsewhere on the same route family.	Minor	Give the complaints-detail 404 state its own copy, e.g. "Support request not found.", instead of sharing a generic not-found component pre-filled with event-review-specific text.	D2_IA03-07_wrong_not_found_message.jpg	2026-08-02
D-007	D2 /complaints/{id}, D3 /dashboard/admin/complaints/{id}, D4 (same URL family as D3)	Usability	No breadcrumb exists anywhere on any support-request detail page (user or admin side), despite each being reached two levels deep (My Requests / Support requests list → request detail). Only a one-step "← Back"/"← Quay lại" link is present, which per the checklist's own distinction cannot express an ancestor path. One finding, three screen instances (D2, D3, D4 all share the same missing-breadcrumb pattern), logged once per the merge rule.	Expected: a breadcrumb exposing the ancestor path on a page reached two levels deep. Actual: no breadcrumb on any request-detail page, user or admin side; only a one-step Back link.	IA03-11 (Slides S13 p.17, Breadcrumb Navigation): "On a page reached through two or more levels, an ancestor path is exposed... Absence of any breadcrumb across a three-level hierarchy... is a Fail, not an N/A." Confirmed on <code>/complaints/25</code> , <code>/complaints/30</code> (user side) and <code>/dashboard/admin/complaints/22</code> , <code>/25</code> , <code>/30</code> (admin side).	3	Add a simple two-segment breadcrumb ("Support requests / #NN") above the "← Back" link on every request-detail page, each segment a real link to its ancestor.	D4_admin_resolved_response_sent.jpg (admin-side request-detail page, shows only the one-step "← Back" link and no breadcrumb; the same shared component renders D2's user-side detail page, which was not separately captured this session)	2026-08-02

ID	Scenario/Screen	Type	Description	Expected vs actual	Steps/Heuristic	Severity	Suggested fix	Evidence (screenshot ref)	Form-submission timestamp
D-008	D3 /dashboard/admin/complaints (default 20; options 10/20/50/100), D2 /complaints (default 10; options 5/10/20)	Bug	The "Rows per page" dropdown cannot actually change its value at all , on either screen. On D3: clicking "10" (verified on-screen coordinates), a <code>form_input</code> DOM value-set (reported success but did not change the rendered value), and a direct <code>element.click()</code> on the live <code><li role="option"></code> node all left the control at "20". On D2 (a different default and option set , but the same underlying component): clicking "20" and separately clicking "5" both left the control at "10", so this is not specific to the value "10" on D3, it is the shared Rows-per-page component failing to apply any selection on any screen it appears on. A workaround exists (next/prev page arrows still work, so the data is not unreachable), which keeps this Major rather than Blocker/Critical. Blocks IA02-12 (keyboard/mouse operability), IA03-06 (pagination-label correctness across a real second page) and IA04-07 (KPI-card totals vs. visible-page count) on D3, since none of those checks can force a smaller page size.	Expected: choosing a value in "Rows per page" changes how many rows the table shows. Actual: the control keeps its default on both screens, under every input method tried.	1. Log in as admin, go to <code>/dashboard/admin/complaints</code> . 2. Click "Rows per page: 20", click "10". 3. Observe: dropdown closes, but "Rows per page" still reads "20". 4. Log in as a student, go to <code>/complaints</code> . 5. Click "Rows per page: 10", click "20" (then separately "5"). 6. Observe: still reads "10" either way.	Major	Instrument the shared rows-per-page <code><Select></code> 's <code>onChange / onSelectChange</code> handler, since it fails identically on two independent instances with different default/option values, look for a bug in the shared component itself (e.g. the handler comparing against a stale prop, or an early-return guard that never actually fires the state update) rather than anything screen-specific.	D3_rows_per_page_stuck_at_20.jpg, D2_rows_per_page_stuck_at_10.jpg	2026-08-02
D-009	D3, /dashboard/admin/complaints	Bug	Switching the Pending/Resolved status filter (clicking either summary-card button) silently clears the currently-active Member-code filter instead of preserving it, confirmed via the URL losing its <code>userMemberCode</code> param the moment the tab switch fires.	Expected: switching the Pending/Resolved tab preserves the active filters. Actual: the member-code filter is cleared and its URL parameter dropped.	1. Log in as admin, go to <code>/dashboard/admin/complaints</code> . 2. Type a real member code (e.g. <code>23127184</code>) into "Member code", URL gains <code>?userMemberCode=23127184...</code> . 3. Click the "Resolved" summary-card button. 4. Observe: URL now reads <code>?tab=resolved</code> only, "Member code" field is empty, and the list shows all resolved requests from every requester, not just the filtered one.	Minor	When the tab-switch handler updates the <code>tab</code> query param, it should preserve all other existing filter params (<code>userMemberCode</code> , <code>category</code> , dates, search) rather than replacing the whole query string.	D3_tab_switch_clears_member_code_filter.jpg	2026-08-02
D-010	D3, /dashboard/admin/complaints	Bug	A specific Category-dropdown interaction (re-opening the dropdown while a previous selection's request was still settling) sent <code>category=undefined</code> to the backend, which correctly rejected it, but the raw validation string ("category must be one of the following values: SUPPORT, COMPLAINT, CONTACT, OTHER") was shown directly to the admin as the on-page error banner, not wrapped in an admin-facing message.	Expected: no category parameter is sent when none is selected, and any validation failure is shown in admin-facing wording. Actual: <code>category=undefined</code> is sent, and the raw backend validation string is rendered as the on-page error banner.	1. Log in as admin, go to <code>/dashboard/admin/complaints</code> , apply a Member-code filter. 2. Open the Category dropdown, select "Complaint", then before its result settles, reopen the dropdown again. 3. Observe the URL gain <code>category=undefined</code> and a red banner reading the raw backend validation string. 4. Selecting a real category value afterwards (e.g. "Support") clears the error and filters correctly, confirming the category filter itself works once given a valid value.	Minor	Client-side, never send <code>category=undefined</code> to the API, omit the param entirely when no category is selected, matching "All categories"; server-side, wrap validation-error responses in a generic user-facing message rather than passing the raw <code>class-validator</code> -style string straight to the UI.	D3_category_undefined_raw_backend_error.jpg	2026-08-02

ID	Scenario/Screen	Type	Description	Expected vs actual	Steps/Heuristic	Severity	Suggested fix	Evidence (screenshot ref)	Form-submission timestamp
D-011	D3, <code>/dashboard/admin/complaints</code> ; D5, <code>/notifications</code>	Bug	Two state-changing admin actions complete successfully while telling the user nothing. (a) D3, Export Excel : clicking it produces zero visible UI feedback of any kind (no busy state, no toast, no completion banner) even though the request succeeds server-side (<code>GET /api/complaints/admin/complaints/export?status=PENDING</code> → 200, confirmed via the Network tab). The export's scope is also silently implicit: it exported only the currently-active status tab's rows, with nothing on screen telling the admin this was a partial, filtered export rather than the whole dataset. (b) D5, Mark all as read : clicking it clears every unread notification — the unread dot and the bell badge both go — while a <code>MutationObserver</code> armed on <code>document.body</code> before the click logged zero additions over a 9 s window and <code>document.querySelectorAll('[aria-live],[role=status],[role=alert]')</code> returned 0 elements. No toast, no banner, no live region: nothing is shown and nothing is announced.	Expected: a visible busy state on click and a completion message stating what happened (and, for the export, what scope was exported). Actual: no visible change of any kind in either case; the export is additionally limited, silently, to the active status tab.	1. Log in as admin, go to <code>/dashboard/admin/complaints</code> (default/Pending tab active). 2. Open DevTools → Network, click "Export Excel". 3. Observe: request appears and succeeds (200), but the page itself shows no busy indicator, toast, or any other acknowledgement that anything happened; the exported query is scoped to <code>status=PENDING</code> with no on-screen statement of that scope. For (b) : 4. Go to <code>/notifications</code> with at least one unread item. 5. Click "Mark all as read". 6. Observe: the unread dot and the header bell badge clear, so the action worked, but no toast, banner or live region ever appears — a screen-reader user gets no confirmation at all, and a sighted user gets only a small dot that vanished while they were looking at the button they just clicked. Confirmed the click hit the right control by capturing <code>event.target</code> (a <code>BUTTON</code> reading "Mark all as read").	Major	Route both actions through the same feedback path the D4 response flow already uses (it raises a <code>role="alert"</code> banner correctly). Add a visible busy state on click plus a completion message: "Export ready, X requests, Pending only." — explicitly stating the scope exported — and "All notifications marked as read." A shared post-action status region would fix both call sites at once, and would stop the next silent action from being written by hand.	(Network-tab evidence only; no distinct on-screen visual to capture since the defect is precisely the <i>absence</i> of any visible change)	2026-08-02
D-012	D1, <code>/complaints/new</code>	Usability	Pressing Enter does nothing (no submit, no navigation) in either the single-line "Vấn đề cần hỗ trợ" input or the multi-line "Mô tả chi tiết" textarea on the create-request form. Logged for the group's awareness rather than as a clear-cut defect: IA02-10 literally expects Enter to trigger the primary submit action on the last field of a multi-field form, but auto-submitting on Enter inside a multi-line description field would itself be a usability regression (users need line breaks), flagging the tension in the checklist item's wording for this screen type, not asserting a shipped bug.	Expected, per IA02-10: Enter triggers the primary submit action from the form fields. Actual: Enter does nothing in either the single-line or the multi-line field. Recorded as a tension in the item wording rather than a shipped defect, since Enter-to-submit inside a description textarea would itself be poor design.	IA02-10 (Norman P4 Mapping / keyboard convention): "Enter triggers the primary submit/search action... and never... a page reload that loses input", the "never loses input" half holds; the "triggers submit" half does not, arguably correctly.	2	If the group intends Enter-to-submit for this screen, wire it explicitly to the single-line field only (not the textarea); if not, consider revising the checklist item's wording to exempt multi-line fields explicitly rather than leaving the tension implicit.	(no screenshot, the defect is the absence of a state change, which a static image cannot usefully show)	2026-08-02

ID	Scenario/Screen	Type	Description	Expected vs actual	Steps/Heuristic	Severity	Suggested fix	Evidence (screenshot ref)	Form-submission timestamp
D-015	D5, <code>/notifications</code> (bell dropdown + full list)	Bug	Every "new complaint" notification (type <code>COMPLAINT_UPDATED</code> , e.g. "Khiếu nại mới từ ...") renders its list-view summary line as <code>Update on complaint "": / Cập nhật khiếu nại ""</code> ; an always-empty complaint title, in both English and Vietnamese. Root cause confirmed via the raw API: this notification type's <code>metadata</code> never includes a <code>complaintTitle</code> field (only <code>complaintId</code> , <code>studentId</code> , <code>studentName</code> , <code>category</code> , <code>legacyType</code>), yet the list-item template reads exactly that missing field. The data is not actually lost: the notification's own <code>content</code> string ("Sinh viên ... đã gửi khiếu nại: Test H") and the dedicated <code>/notifications/{id}</code> detail page both show the correct complaint reference, only the list-view summary line is broken. The same notification's category badge is separately mislabelled "Event update" for what is a Complaint-type item, consistent with the same underlying type-mapping never being finished for this notification type.	Expected: the list-view summary shows the complaint title, as the sibling notification type already does. Actual: it shows empty quotes, in both languages, and the category badge reads "Event update" on a Complaint-type item.	1. Log in as admin, open the bell dropdown or <code>/notifications</code> . 2. Find any "Khiếu nại mới từ ..." row. 3. Observe "Update on complaint "":" with empty quotes. 4. Toggle EN/VI: the same empty-quotes bug persists as "Cập nhật khiếu nại """, ruling out a translation-only issue. 5. Click "... " → "View details": the detail page's "Content" field shows the correct, non-empty complaint reference.	Major	Populate <code>complaintTitle</code> in this notification type's metadata at creation time (matching what <code>COMPLAINT_REPLIED</code> already does correctly), or have the list-item template fall back to parsing the already-correct <code>content</code> string when <code>complaintTitle</code> is absent; fix the category-badge label mapping for this type in the same pass.	D5_dropdown_resting.jpg, D5_notification_detail_correct_content.jpg, D5_i18n_vietnamese_full.jpg (the empty <code>""</code> quotes visible on the "Khiếu nại mới từ..." row)	2026-08-02
D-016	D5, header notification bell dropdown	Bug	Pressing Escape does not close the header notification dropdown. The panel stays fully open and is visually identical before and after, and repeated presses make no difference: re-verified on 2026-07-31 by pressing Escape three times in succession with the panel open. The dropdown's own "X" control closes it correctly and is keyboard-focusable, and clicking outside the panel also closes it, so only the Escape shortcut is missing. Scope deliberately limited to this one component. The attachment image lightbox (D6) was tested for the same defect and does not exhibit it: Escape closes the lightbox on the first press, confirmed on complaints 25 and 26. An earlier version of this finding claimed both overlays were affected and attributed them to one shared handler; the live re-verification refuted that, and the negative result is recorded here so the finding is not read more broadly than the evidence supports.	Expected: Escape dismisses the open notification dropdown, the convention the product's other overlay (the attachment lightbox) already follows. Actual: the dropdown stays fully open, on the first press and on every subsequent press.	1. Log in, click the header bell icon to open the notification dropdown. 2. Press Escape. 3. Observe: the panel remains fully open, identical before and after. 4. Press Escape twice more: still open. 5. Click the visible X, or click outside the panel: closes correctly either way, confirming only the Escape path is affected.	Minor	Add an Escape handler to the notification dropdown, bound at the document level while the panel is mounted, matching the behaviour the attachment lightbox already has. Rated Minor rather than higher because a keyboard user can still Tab to the close control and activate it, and because the equivalent overlay elsewhere in the product behaves correctly, so this is a single missing handler rather than a product-wide convention gap.	D5_esc_does_not_close_dropdown.jpg	2026-08-02
D-017	D5, <code>/notifications</code>	Bug	Clicking "... " → "Delete" on a notification removes it from the list immediately , with no confirmation dialog, no "this cannot be undone" wording, and no undo affordance of any kind.	Expected: a confirmation step or an undo window before a destructive delete. Actual: the row disappears the instant Delete is clicked, with no intermediate step and no way back.	1. Log in, open <code>/notifications</code> . 2. Click "... " on any notification row, click "Delete". 3. Observe: the row disappears instantly with zero intermediate step.	Minor	Add a confirmation step (dialog, or an "Undo" toast with a short window) before or immediately after a destructive delete, consistent with IA04-03's expectation for destructive actions elsewhere in the product.	D5_delete_no_confirmation.jpg	2026-08-02

ID	Scenario/Screen	Type	Description	Expected vs actual	Steps/Heuristic	Severity	Suggested fix	Evidence (screenshot ref)	Form-submission timestamp
D-019	D5 /notifications, D2 /complaints/{id}	Bug	The browser tab <title> stays in Vietnamese even after switching the language toggle back to English, while every visible in-page label is already correctly in English at the same moment. Confirmed on /notifications ("Thông báo HCMUS EMS") via a direct document.title read taken at the same instant as the English-rendered screenshot, and independently on /complaints/{id} ("Chi tiết yêu cầu HCMUS EMS") on live re-verification (2026-07-31), where the page body was fully English but the tab title stayed Vietnamese. Two independent routes sharing the same defect, one finding.	Expected: the browser tab title follows the active language, as every in-page label does. Actual: on /notifications it stays "Thông báo HCMUS EMS"; on /complaints/{id} it stays "Chi tiết yêu cầu HCMUS EMS"; both while the page body is fully English.	1. Log in, go to /notifications . 2. Toggle to Vietnamese, then back to English. 3. Read document.title in DevTools: still "Thông báo HCMUS EMS" despite the page body being fully English. 4. Repeat on /complaints/{id} : document.title stays "Chi tiết yêu cầu HCMUS EMS" under the same conditions.	Trivial	These routes' <title> are likely set once server-side rather than reading the active locale on client-side language switches; wire both to the same i18n state the rest of the page already uses correctly.	D5_title_tag_stuck_vietnamese.jpg (the /notifications instance. The second route, /complaints/{id} , was confirmed on live re-verification (2026-07-31) by reading document.title against a page body that was already fully English, and was not separately captured; the finding's severity and description rest on the captured instance, with the second route recorded as a corroborating observation rather than as separately evidenced)	2026-08-02
D-020	D3 /dashboard/admin/complaints, D4 /dashboard/admin/complaints/{id}	Bug	The admin area has no responsive breakpoint for phone-width viewports. The left navigation sidebar keeps its full desktop width, leaving roughly a quarter of the screen for content: the page heading wraps one word per line and the results table is pushed off-screen. Collapsing the sidebar by hand restores a usable layout, which is what identifies the fault as the default state rather than the content. Found during Task 3 cross-platform execution on 2026-08-02.	Expected: below a phone-width breakpoint the sidebar collapses to icons (or off-canvas) automatically, as it does when collapsed manually. Actual: it renders at desktop width regardless of viewport, and the user must discover the collapse control before the page is readable.	Open /dashboard/admin/complaints as an admin on any phone-width device — load the URL fresh ; navigating from another admin screen inherits that screen's collapsed state and hides the defect. Repeat for /dashboard/admin/complaints/{id} . Cause is isolated by elimination across the matrix: reproduced on Blink (Android 16 / Chrome, Galaxy S23 FE, 1080 px) and WebKit (iOS 26.5 / Safari, iPhone 15, 1179 px), so not engine-specific; the same Android 16 + Blink combination on a Redmi Pad 2 tablet renders correctly, so not OS-specific; D1 and D2 pass on both handsets, so not small screens in general.	Major	Add a viewport breakpoint to the admin layout that auto-collapses the sidebar below roughly tablet width. The threshold to target lies between 1179 px (fails) and the Redmi Pad 2's width (passes).	reports/evidence_task3/D3_Android_Chrome_phone.png , D4_Android_Chrome_phone.png , D3_iOS_Safari_phone.png , D4_iOS_Safari_phone.png ; passing controls: D3_Android_Opera_tablet.png , D4_Android_Opera_tablet.png	2026-08-02
D-021	D1 /complaints/new, D2 /complaints	Bug	On Safari 15 (macOS Monterey 12) the application's stylesheet does not apply at all. The page falls back to browser defaults: serif type, unstyled native form controls, blue underlined links, and the logo at intrinsic size overflowing the viewport. The same two screens on Safari 18 render perfectly, so this is specific to the older build rather than to WebKit. Reproduced in a second independent Sauce Labs session before being logged.	Expected: the same rendering as every other tested browser, or a graceful degradation that is still usable. Actual: the app is delivered with no styling whatsoever, leaving a raw HTML document.	Open prod-dev.ems-fitus.cloud/complaints/new in Safari 15 on macOS Monterey 12 (Sauce Labs Live). Sign in; the failure is visible before and after authentication. Compare against Safari 18 on macOS Sequoia, where the same URL renders correctly.	Major	Identify which CSS feature in the bundle Safari 15 cannot parse — a single unsupported at-rule or syntax error can cause a whole stylesheet to be dropped. Either add a fallback or set an explicit minimum supported Safari version and tell users.	reports/evidence_task3/D1_macOS_Safari15_desktop.png , D2_macOS_Safari15_desktop.png , reports/evidence_task3/safari15_defect/safari15_login_unstyled_repro.jpg	2026-08-02

ID	Scenario/Screen	Type	Description	Expected vs actual	Steps/Heuristic	Severity	Suggested fix	Evidence (screenshot ref)	Form-submission timestamp
D-022	Sign-in screen (/login) — reached from D1	Bug	On Safari 16 (macOS Monterey 12) the CSS does apply, but the Email and Password input fields never render. Only the Login button and the "Forgot password?" / "Create guest account" links appear. A user on Safari 16 therefore cannot sign in at all, which makes every screen behind authentication unreachable. Verified across three captures spanning about two minutes with no change; a scrollbar is present, so the DOM has laid out content that is not being painted — this is a render fault, not an unfinished load. Reproduced in a second, independent Sauce Labs session on 2026-08-02 (new session id, EU-Central-1 data centre, Safari 16 / macOS Monterey 12 selected fresh), and within that session it survived a full page reload — the reloaded page renders the identical button-and-links-only layout, which is what rules out a one-off partial paint.	Expected: the sign-in form renders with its two inputs, as on every other tested browser. Actual: the inputs are absent while their surrounding chrome renders, and the form cannot be completed.	Open prod-dev.ems-fitus.cloud/login in Safari 16 on macOS Monterey 12 (Sauce Labs Live) and wait; the fields do not appear. Reload the page: they still do not appear. Contrast with Safari 15 (everything renders, merely unstyled) and Safari 18 (correct). Reproduced in two independent sessions.	Critical	Investigate the input components' rendering path on Safari 16 specifically — the failure sits between Safari 15's total CSS drop and Safari 18's correct render, so it is a distinct fault, not a milder form of D-021.	Session 1: reports/evidence_task3/safari15_defect/safari16_login_fields_missing.jpg, safari16_login_fields_missing_2min.jpg. Session 2 (2026-08-02): safari16_login_fields_missing_session2.jpg, and after a full reload safari16_login_fields_missing_session2_reload.jpg	2026-08-02
D-023	D5 /notifications; D6 attachment lightbox (opened from D4)	Usability	Client-side view state is never written to the URL or the history stack, so the browser Back button does the wrong thing in two places. (a) D5: paging the notifications list leaves the address bar as a bare /notifications on every page. Going to page 2, opening a notification's record and pressing Back returns the list to page 1 ; the user's position is gone. (b) D6: opening the attachment lightbox pushes no history entry (history.length unchanged, 8 before and 8 after), so Back does not close the full-viewport overlay — it leaves the record entirely and lands on whatever unrelated page came earlier.	Expected: Back returns the user where they were — the list still on the page they had reached, and a full-screen overlay closed rather than the record abandoned. Actual: (a) the list silently resets to page 1; (b) Back discards both the overlay and the record being viewed.	Nielsen H3 (User Control and Freedom). For (a): 1. Sign in as admin, open /notifications with more than 10 notifications so a second page exists. 2. Click page 2 — note the address bar does not change. 3. Open the notification's linked record. 4. Press browser Back. 5. Observe the list renders page 1, not page 2. For (b): 6. Open a support request with an attachment, e.g. /dashboard/admin/complaints/24. 7. Click a thumbnail to open the lightbox. 8. Press browser Back. 9. Observe the overlay does not close; the browser leaves the record. Checklist item IA03-13 on both screens.	2	Put the list's page (and rows-per-page) in the query string the way the admin list at /dashboard/admin/complaints already does — it writes ?search=...&page=... and consequently passes this same check — so history restores it for free. For the lightbox, push a history entry when it opens and close it on popstate, so Back closes the overlay instead of leaving the page.	(No capture: both halves are about what the address bar and history do, which a page screenshot cannot show; the D5 half is reproducible in four steps above and the D6 half was measured via history.length)	2026-08-03
D-024	D1 /complaints/new, reached from the event pages	Usability	The support entry point is not discoverable from where users actually look for it. In moderated usability sessions on 2026-08-03, 2 of 5 participants began the "report a registration problem" task somewhere other than Support, and neither reached the form by recognising it — both arrived by elimination after the pages they expected offered no route onward. The entry point is named after the organisation's internal category ("Hỗ trợ") rather than after the user's situation; a registration problem is mentally filed under the event, and the event page offers no way onward.	Expected: a user who has a problem with an event registration can reach the reporting form from the event context, or can recognise where to go on first look. Actual: the route is discovered by trial and error across Hồ sơ / Sự kiện / Hỗ trợ.	Nielsen H2 (Match between system and the real world), H6 (Recognition rather than recall). 1. Give a user who has not seen EMS the goal "you registered for an event but check-in says you did not; report this". 2. Do not name any menu. 3. Observe which section they open first. Participants P2 and P4 (of 5) opened event/profile pages before Support. P2: "Ban đầu tôi tìm trong trang Sự kiện vì nghĩ vấn đề đăng ký sẽ nằm ở đó." P4: "Tôi không chắc nên tìm ở Hồ sơ, Sự kiện hay Hỗ trợ."	3	Add a "Bảo vấn đề về đăng ký này" action in the event / registration context that deep-links to /complaints/new with the event pre-filled. At minimum, surface Support from the event page rather than only from the global menu.	reports/evidence_task2/Session_P2.mp4, reports/evidence_task2/Session_P4.mp4	2026-08-03

ID	Scenario/Screen	Type	Description	Expected vs actual	Steps/Heuristic	Severity	Suggested fix	Evidence (screenshot ref)	Form-submission timestamp
D-025	D2 /complaints (list, immediately after submitting on D1)	Usability	After filing a request, the user cannot tell which row in the list is the one they just created. The list gives no "just submitted" cue — no highlight, no anchor, no recency marker — so identifying your own newest request means reading rows and reasoning from dates and titles. In the 2026-08-03 sessions one participant opened the wrong request and recovered only by comparing dates and titles; a second described the same search as slow. Default ordering does not communicate recency to a user who does not already know the ordering.	Expected: after a state-changing action the user is returned somewhere that continues from what they just did. Actual: they are returned to an undifferentiated list and must re-find their own request.	Nielsen H1 (Visibility of system status), H6 (Recognition rather than recall). 1. Submit a support request on /complaints/new. 2. Observe where the application lands. 3. From that list, identify the request just submitted without using the timestamp column. Participant P4: "ở danh sách tôi cũng không biết dòng nào là yêu cầu vừa gửi", "tôi mở nhầm yêu cầu cũ rồi dựa vào ngày và tiêu đề để quay lại". P2: "tìm lại yêu cầu hơi lâu."	3	Redirect to /complaints/{id} — the detail page of the request just created — instead of the list. If the list must remain the landing page, highlight and anchor to the new row. This single change also resolves D-027.	reports/evidence_task2/D-025_P4_two_pending_rows_same_timestamp.png (P4's own list: two Pending requests, both stamped Aug 3, 2026, 1:40 PM, nothing marking which is his), reports/evidence_task2/Session_P4.mp4	2026-08-03
D-026	D1 /complaints/new, request-type control	Usability	The request-type options carry no explanation, so users cannot map their situation onto one — and fear that changing the choice will discard what they have already typed. Two of five participants stalled at this same control on 2026-08-03. One was reasoning about categories, the other about data loss, which is a second-order cost of the same uncertainty.	Expected: each request type is recognisable from its label or from an example, and changing it visibly preserves entered content. Actual: unexplained labels, and no reassurance that switching is safe.	Nielsen H2 (Match between system and the real world), H5 (Error prevention). 1. Open /complaints/new as a user who has not used EMS. 2. Read the request-type options and choose one for "registered but check-in says I am not". 3. Change the selection after typing a description and observe whether the description survives. Participant P1: "Ở phần chọn loại yêu cầu tôi dừng lại một chút vì chưa chắc trường hợp mất đăng ký thuộc mục nào." P5: "tôi phân vân giữa hai loại yêu cầu và không chắc đổi lựa chọn có làm mất nội dung đã nhập không."	2	Add one line of example text under each request type (e.g. "ví dụ: đã đăng ký nhưng không check-in được"). Preserve entered content across a type change and make that visibly true.	reports/evidence_task2/D-026_P2_request_type_options_unexplained.png — the open dropdown showing all four options as bare labels (Support · Complaint · Contact · Other) with no example or description; also reports/evidence_task2/Session_P1.mp4, Session_P5.mp4	2026-08-03
D-027	D1 /complaints/new, post-submit confirmation	Usability	The submission confirmation is too transient to establish trust in a state-changing action. 3 of 5 participants on 2026-08-03 did not believe the request had been filed until they navigated elsewhere and saw it persisted. Trust was carried by persistent artefacts the confirmation never shows — the request ID and the status pill — while the toast itself disappeared before it had done its job.	Expected: after submitting, the user is confident the request exists, without having to go and verify it. Actual: the only acknowledgement is a time-limited toast; confidence arrives later, from the list or the detail page.	Nielsen H1 (Visibility of system status). 1. Submit a support request on /complaints/new. 2. Without navigating, note what remains on screen after the toast dismisses. 3. Observe that no request ID or status is visible anywhere on the resulting page. Participant P4: "Tôi chưa thật sự tin ngay sau khi gửi vì thông báo biến mất nhanh; chỉ khi mở được chi tiết và thấy trạng thái hoặc phản hồi tôi mới yên tâm." P2: "có thông báo đã gửi nhưng chỉ yên tâm hơn khi thấy yêu cầu nằm trong danh sách."	2	Replace the transient toast with a persistent confirmation naming the request ID and its status, on the page the user lands on after submitting. Fixing D-025 by redirecting to the new request's detail page delivers this at the same time.	reports/evidence_task2/Session_P4.mp4	2026-08-03

Note on D-022's severity. Blocking sign-in is Critical by impact. When first written this finding rested on a single cloud session, and it was logged at Critical with that limitation stated rather than quietly downgraded or quietly inflated. **That gap is now closed:** a second independent session on 2026-08-02 reproduced it, and a full page reload inside that session reproduced it again, so Task 3's own rule — reproduce a failure in a fresh session before filing it as final — is satisfied for D-022 exactly as it was for D-021. What the evidence still does not establish is breadth: both sessions ran the same Sauce Labs Safari 16 image, so this is a confirmed fault on that build, not a measured statement about every Safari 16 in the wild.

Note on why D-020 is one finding and not four. Four matrix cells fail (D3 and D4 on two separate phones), but they share a single root cause, and the log's own rule is to group by cause rather than by symptom. Filing four thin findings would inflate the count while making the actual defect harder to see; the four cells are carried as evidence on the one finding instead.

Retracted findings

Three findings were withdrawn before any Google Form submission was made, so nothing needs reconciling on the form side. Each ID is retired rather than reassigned, under the rule that a finding ID is never reused. They are kept visible here rather than deleted, because what was claimed and why it did not survive is part of the record.

D-013 retracted (2026-07-31), on live re-verification. Originally logged as a Usability 1: the attachment image lightbox needed two Escape presses to close on the admin side (D3 and D4) against one on the user side (D2), which was written up as a timing inconsistency in the Escape listener. Re-tested against the live system on the student's authenticated admin session: the lightbox was opened on complaint 25 and Escape pressed with focus untouched, and it closed on the **first** press. It was reopened, the image inside the modal was clicked, Escape was pressed again, and it closed on the first press once more. On complaint 26 the lightbox likewise closed on the first press. The two-press behaviour did not reproduce under any of these conditions. A single unreproducible observation is not a finding.

D-014 retracted (2026-07-31), on self-review of the log. Originally logged as: the Requester-information panel showing "Tôi là Admin" instead of a real name. On review, `<p class="...">Tôi là Admin</p>` is a single flat text node, not a static "Tôi là {role}" template wrapping a separate name field, so this is almost certainly just the literal display name this test/seed admin account was actually given (a quirky but genuine value), not a UI-label string that leaked into a data field. The live re-verification supports this: on complaint 25 the same panel renders "DUYÊN LÊ PHẠM KIỀU" with the correct email and member code, so the field does display a real name whenever the account has one.

D-018 retracted (2026-07-31), on live re-verification. This was the most severe finding in the log, rated Critical, and it was wrong. Originally logged as: on complaint 26 the attachment lightbox opens but the image content never renders, with the supporting claims that "zero network requests were ever issued for the attachment" and that "no `<img>` element matching the expected content exists in the DOM", from which the conclusion was drawn that the source URL never reaches the render step.

Both of those claims are factually wrong. The re-verification found the `<img>` element present in the DOM with `alt="attachment_1"`, its `src` pointing at `/data/event-storage/uploads/complaints/17/...-d1-test-upload.png`, and the browser reporting `complete: true`. Fetching that URL directly returns **HTTP 200**, `content-type: image/png`, and a body of **68 bytes** whose PNG header decodes to `IHDR width=1 height=1`. The stored file is a 1 by 1 pixel PNG. The lightbox was rendering it correctly the whole time; a 1 by 1 image scaled into a large pane simply looks blank. The filename indicates a synthetic placeholder uploaded during the D1 upload test rather than real user evidence, so the blank pane traces to the test data, not to the product. No defect remains.

Column guide

The table above carries the nine columns §7 requires, plus **Expected vs actual**, which §6 Part B requires for every bug report.

- **ID:** stable and never reused, in the form `D-001`, `D-002` and so on. The `D` prefix is the scenario, which keeps these IDs distinguishable from teammates' A, B and C logs if the group merges them.
- **Scenario/Screen:** `D1` through `D6`, or the usability-session task or cross-platform cell the finding came from.
- **Type:** `Bug` (behaves contrary to spec, or breaks) or `Usability` (works, but costs the user). Each type is rated on its own severity scale, below.
- **Description:** one or two sentences, quoting the application's own strings exactly.
- **Expected vs actual:** the behaviour the checklist item or the product's own convention leads a user to expect, set against what the screen actually did.
- **Steps/Heuristic:** for a `Bug`, numbered reproduction steps from a known state (URL, account, role, precondition). For a `Usability` finding, the heuristic or checklist item it violates, for example `IA04-14`

or Nielsen H1.

- **Severity:** for a **Bug**, Blocker / Critical / Major / Minor / Trivial. For a **Usability** finding, Nielsen 0 to 4. The two scales are never blended in one column.
- **Suggested fix:** a specific, arguable change rather than a general aspiration.
- **Evidence (screenshot ref):** a filename that exists under `reports/evidence_task1b/`, `reports/evidence_task2/`, or `reports/evidence_task3/`. Where a defect is the *absence* of any visible change, the cell says so and cites the DevTools evidence instead.
- **Form-submission timestamp:** when the finding was submitted to the \$7 Google Form. **All 19 were submitted on 2026-08-02**, in the packet order D-001 → D-022, from the student-ID address `lpkduyen23@clc.fitus.edu.vn` — the same address burned into the Task 3 screenshot overlays, so a cross-check between the form responses and this file resolves to one person. **Recorded to the day, not the minute**, because per-submission clock times were not written down as the nineteen were sent. The date is stated rather than a made-up time: the form keeps its own exact timestamp for every response, which is the authoritative record if a minute-level comparison is ever needed.

Severity scales

The two scales below are never mixed in one column.

Bug: impact on function:

	Meaning
Blocker	Core flow cannot be completed; no workaround
Critical	Major function broken or data lost; workaround is painful
Major	Function impaired, workaround exists
Minor	Small deviation, easily worked around
Trivial	Cosmetic, no functional effect

Usability: Nielsen 0-4, judged on frequency + impact + persistence together:

	Meaning
0	Not a usability problem
1	Cosmetic, fix if time permits
2	Minor, low priority
3	Major, important to fix, high priority
4	Catastrophe, imperative to fix before release

An accessibility defect that removes the *only* path for a group of users (for example a modal that ignores ESC for a keyboard-only user) is rated from the barrier it creates, not from how many people it blocks: Major, or usability severity 3, at minimum.

Cross-cutting causes

- **Shared-component regressions rather than per-screen bugs.** D-005 (search box drops keystrokes) and D-008 (rows-per-page cannot be changed) each surfaced identically on two different screens with two different data/option configurations, strong evidence that the fix belongs in one shared component (a search-input and a `<Select>`), not in per-screen code. D-008's D5 counter-example (the same *kind* of rows-per-page control works correctly on `/notifications`) further narrows the fault to the specific list-table component D2/D3 both use, not every instance of the control in the product.

- **A stale-closure/timing pattern recurring across unrelated inputs.** D-001 (Request-type dropdown reverting on a fast next click) and D-005 (search box losing keystrokes on fast typing) both look, mechanically, like a handler capturing a stale value inside a closure/debounce that a fast subsequent event outruns. Worth a single audit pass across the app's input/debounce utilities rather than fixing each call site separately.
- **One overlay missing an Escape handler, not a product-wide convention gap, and the difference only became clear on re-verification.** D-013 and D-016 were originally logged as two symptoms of one shared handler bound inside the dialog subtree, an explanation that fitted the notes and was wrong. Re-testing both overlays live showed the attachment lightbox closes on the first Escape press under every condition tried, while the notification dropdown ignores Escape entirely. So D-016 is a single missing handler on one component, D-013 does not reproduce at all, and the product does have a working Escape convention that this one dropdown is outside of. The generalisable point is about method rather than about the product: a root cause that unifies two findings is an attractive story, and attractive stories need testing before they are written down.
- **Test data can look exactly like a defect.** D-018 was rated Critical on the strength of a blank image pane, and the stored file turned out to be a valid 68 byte, 1 by 1 pixel PNG that the product rendered correctly. Nothing on screen distinguishes "the viewer is broken" from "the file is one pixel", which is worth noting as a genuine, if minor, observability gap. It is not logged as a finding, because the only observed instance was a placeholder the testing itself uploaded.
- **Destructive actions with no confirmation step, product-wide.** D-017 (delete a notification) joins a pattern already implicit in IA04-03's own scope, worth checking whether other quiet-delete affordances exist elsewhere (row-level deletes on Users Management, Events Management) that this session did not re-test.
- **Notification metadata that was only finished for one of two message types.** D-015's empty-title bug and its sibling mislabelled category badge both stem from the same root cause: `COMPLAINT_UPDATED` notifications were never given the same metadata completeness as `COMPLAINT_REPLIED` notifications, even though both are rendered by the same list-item template. One field-population fix (or one template fallback) closes both symptoms at once.